

Notes on Understanding Black-box Predictions via Influence Functions

Prepared by Anushka Mathur

09/04/2026

1 Introduction

- The goal is to understand the predictions of a black-box model by tracing them back to the training data.
- Explicitly removing a training point and retraining the model to observe the change (leave-one-out retraining) is **computationally expensive**.
- **Solution:** We use Influence Functions to mathematically approximate this perturbation without retraining.

2 Approach

Let the input space be X and the output space be Y . We are given training points $z_1, \dots, z_n$, where $z_i = (x_i, y_i) \in X \times Y$.

Let $L(z, \theta)$ be the loss function. The optimal parameters $\hat{\theta}$ minimize the empirical risk:

$$\hat{\theta} \triangleq \arg \min_{\theta \in \Theta} \frac{1}{n} \sum_{i=1}^n L(z_i, \theta) \quad (1)$$

Assumption: The empirical risk is strictly convex and twice differentiable.

Upweighting a Training Point

Instead of removing a point z , we calculate the parameter update after changing the weight of z by a small amount ϵ :

$$\hat{\theta}_{\epsilon, z} \triangleq \arg \min_{\theta \in \Theta} \frac{1}{n} \sum_{i=1}^n L(z_i, \theta) + \epsilon L(z, \theta) \quad (2)$$

- **Influence on Parameters:**

$$\mathcal{I}_{up, params}(z) = \left. \frac{d\hat{\theta}_{\epsilon, z}}{d\epsilon} \right|_{\epsilon=0} = -H_{\hat{\theta}}^{-1} \nabla_{\theta} L(z, \hat{\theta}) \quad (3)$$

where $H_{\hat{\theta}} \triangleq \frac{1}{n} \sum_{i=1}^n \nabla_{\theta}^2 L(z_i, \hat{\theta})$ is the Hessian matrix.

- **Influence on Test Loss (using Chain Rule):**

$$\mathcal{I}_{up,loss}(z, z_{test}) = \left. \frac{dL(z_{test}, \hat{\theta}_{\epsilon, z})}{d\epsilon} \right|_{\epsilon=0} = -\nabla_{\theta} L(z_{test}, \hat{\theta})^{\top} H_{\hat{\theta}}^{-1} \nabla_{\theta} L(z, \hat{\theta}) \quad (4)$$

Perturbing a Training Point

We can also measure the effect of modifying a training input x slightly. Let $z_{\delta} = (x + \delta, y)$. For continuous inputs where $\|\delta\| \rightarrow 0$, the change in test loss is:

$$\mathcal{I}_{pert,loss}(z, z_{test}) = \left. \nabla_{\delta} L(z_{test}, \hat{\theta}_{z_{\delta}, -z}) \right|_{\delta=0} = -\nabla_{\theta} L(z_{test}, \hat{\theta})^{\top} H_{\hat{\theta}}^{-1} \nabla_x \nabla_{\theta} L(z, \hat{\theta}) \quad (5)$$

This helps identify which specific features of z are most responsible for the prediction on z_{test} .

3 Comparison: Relation to Euclidean Distance

Why not just use Euclidean distance (maximizing $x \cdot x_{test}$) to find influential points?

- Because it only measures superficial visual similarity.
- Influence functions solve this by measuring how a specific data point actually altered the model's parameters, capturing its true impact.

Consider a logistic regression model where the probability is given by $p(y|x) = \sigma(y\theta^{\top}x)$, with labels $y \in \{-1, 1\}$ and σ representing the sigmoid function.

For a given training point $z = (x, y)$, the loss function is:

$$L(z, \theta) = \log(1 + \exp(-y\theta^{\top}x)) \quad (6)$$

The gradient of the loss with respect to the parameters is:

$$\nabla_{\theta} L(z, \theta) = -\sigma(-y\theta^{\top}x)yx \quad (7)$$

The Hessian matrix is defined as:

$$H_{\theta} = \frac{1}{n} \sum_{i=1}^n \sigma(\theta^{\top}x_i) \sigma(-\theta^{\top}x_i) x_i x_i^{\top} \quad (8)$$

Using these components, the influence on the test loss $\mathcal{I}_{up,loss}(z, z_{test})$ evaluates to:

$$-y_{test}y \cdot \sigma(-y_{test}\theta^{\top}x_{test}) \cdot \sigma(-y\theta^{\top}x) \cdot x_{test}^{\top} H_{\hat{\theta}}^{-1} x \quad (9)$$

Comparing this result to the simple Euclidean inner product $x \cdot x_{test}$, we observe two key differences:

- **High training loss has more influence:** The term $\sigma(-y\theta^{\top}x)$ scales the influence, giving points with higher training loss greater weight.
- **Weighted covariance matrix measures resistance:** The inverse Hessian $H_{\hat{\theta}}^{-1}$ acts as a weighted covariance matrix. It measures the resistance of the other training points to the removal of z .

4 Efficiently Calculating Influence

The Problem: Inverting the Hessian matrix explicitly takes $\mathcal{O}(np^2 + p^3)$ time, which is far too slow for large models.

The Solution: Avoid forming the inverse Hessian entirely by using **Hessian-Vector Products (HVPs)**.

Step 1: Approximate the intermediate vector:

$$s_{test} \stackrel{\text{def}}{=} H_{\hat{\theta}}^{-1} \nabla_{\theta} L(z_{test}, \hat{\theta})$$

Step 2: Efficiently compute the influence using the result:

$$\mathcal{I}_{up,loss}(z, z_{test}) = -s_{test} \cdot \nabla_{\theta} L(z, \hat{\theta})$$

Conjugate Gradients

- Frames the inversion as an optimization problem: $\arg \min_t \{t^{\top} H_{\hat{\theta}} t - v^{\top} t\}$.
- Drastically reduces time complexity to $\mathcal{O}(np)$.

Stochastic Estimation

- Relies on a Taylor expansion.
- Samples t training points and repeats the process r times.
- Recursively computes the following equation:

$$\hat{H}_j^{-1} v = v + (I - \nabla_{\theta}^2 L(z_{s_j}, \hat{\theta})) \hat{H}_{j-1}^{-1} v$$

- Requires an initial state defined as $\hat{H}_0^{-1} v = v$.
- Operates in $\mathcal{O}(np + rtp)$ time.

Takeaways:

- This identical HVP trick also extends to calculating perturbation influence ($\mathcal{I}_{pert,loss}$).
- The process can be readily implemented in standard frameworks like TensorFlow.

Validation and Extension

In practice, models often violate the initial assumptions:

- **Non-Convex Models:** If the Hessian has negative eigenvalues, we apply a convex quadratic approximation by adding a damping term λI .
- **Non-Differentiable Losses:** For losses whose derivatives do not exist (like Hinge loss), we substitute a smooth approximation (like SmoothHinge) to compute influence accurately.

5 Uses of Influence Functions

Influence functions are versatile and can be applied to several real-world debugging tasks:

- **Understanding Model Behavior:** Reveals exactly which training points are “responsible” for a specific prediction.
- **Adversarial Training Examples:** Shows how a maximum influence perturbation on a small number of training points can flip a test prediction.
- **Debugging Domain Mismatch:** Identifies the root cause when differing test and train distributions lead to poor results.
- **Fixing Mislabelled Examples:** Allows human reviewers to fix only the most important mislabels by prioritizing highly influential points.

Derivation

$\hat{\theta}$ minimizes Empirical Risk

$$R(\theta) \stackrel{\text{def}}{=} \frac{1}{n} \sum_{i=1}^n L(z_i, \theta)$$

L is twice-differentiable and convex.

$$H_{\hat{\theta}} \stackrel{\text{def}}{=} \nabla^2 R(\hat{\theta}) = \frac{1}{n} \sum_{i=1}^n \nabla_{\theta}^2 L(z_i, \hat{\theta})$$

exists and is l.t. $H_{\hat{\theta}}^{-1}$

$$\hat{\theta}_{\epsilon, z} = \arg \min_{\theta \in \Theta} \{R(\theta) + \epsilon L(z, \theta)\}.$$

$$\Delta_{\epsilon} = \hat{\theta}_{\epsilon, z} - \hat{\theta}$$

$$\frac{d\hat{\theta}_{\epsilon, z}}{d\epsilon} = \frac{d\Delta_{\epsilon}}{d\epsilon}$$

$$0 = \nabla R(\hat{\theta}_{\epsilon, z}) + \epsilon \nabla L(z, \hat{\theta}_{\epsilon, z})$$

$\hat{\theta}_{\epsilon, z} \rightarrow \hat{\theta}$ as $\epsilon \rightarrow 0 \rightarrow$ Taylor expansion

$$0 \approx \left[\nabla R(\hat{\theta}) + \epsilon \nabla L(z, \hat{\theta}) \right] + \left[\nabla^2 R(\hat{\theta}) + \epsilon \nabla^2 L(z, \hat{\theta}) \right] \Delta_{\epsilon},$$

dropping higher terms.

$$\text{solve for } \Delta_{\epsilon}, \quad \Delta_{\epsilon} \approx - \left[\nabla^2 R(\hat{\theta}) + \epsilon \nabla^2 L(z, \hat{\theta}) \right]^{-1} \left[\nabla R(\hat{\theta}) + \epsilon \nabla L(z, \hat{\theta}) \right]$$

$\hat{\theta}$ minimizes $R(\hat{\theta})$, hence, $\nabla R(\hat{\theta}) = 0$ dropping $\mathcal{O}(\epsilon)$ terms.

$$\Delta_{\epsilon} \approx - \nabla^2 R(\hat{\theta})^{-1} \nabla L(z, \hat{\theta}) \epsilon$$

$$\therefore \left. \frac{d\hat{\theta}_{\epsilon, z}}{d\epsilon} \right|_{\epsilon=0} = -H_{\hat{\theta}}^{-1} \nabla L(z, \hat{\theta}) \stackrel{\text{def}}{=} \mathcal{I}_{\text{emp, param.}}(z).$$

$\hat{\theta}$ is not local minimum.

$$g \stackrel{\text{def}}{=} \frac{1}{n} \sum_{i=1}^n \nabla_{\theta} L(z_i, \hat{\theta})$$

$g \neq 0$

upweight z by ϵ

$$g \mapsto g + \epsilon \nabla_{\theta} L(z, \hat{\theta}) \quad H_{\hat{\theta}} \mapsto H_{\hat{\theta}} + \epsilon \nabla_{\theta}^2 L(z, \hat{\theta})$$

$$N_{\epsilon, z} \stackrel{\text{def}}{=} - \left[H_{\hat{\theta}} + \epsilon \nabla_{\theta}^2 L(z, \hat{\theta}) \right]^{-1} \left[g + \epsilon \nabla_{\theta} L(z, \hat{\theta}) \right]$$

$$N_{\epsilon, z} \approx -H_{\hat{\theta}}^{-1} \left(g + \epsilon \nabla_{\theta} L(z, \hat{\theta}) \right)$$